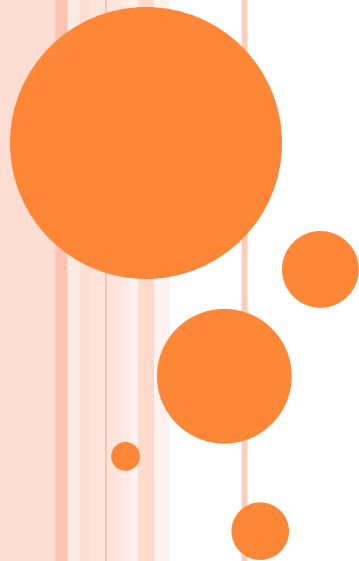


PROCEDURES

Computer Organization and Assembly Language

Computer Science Department

National University of Computer and Emerging
Sciences Islamabad

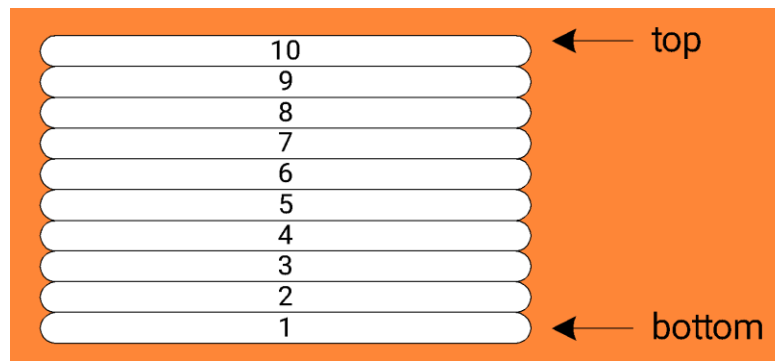


AGENDA FOR TODAY

- Stack Operations
- Defining and Using Procedures

Imagine a stack of plates . . .

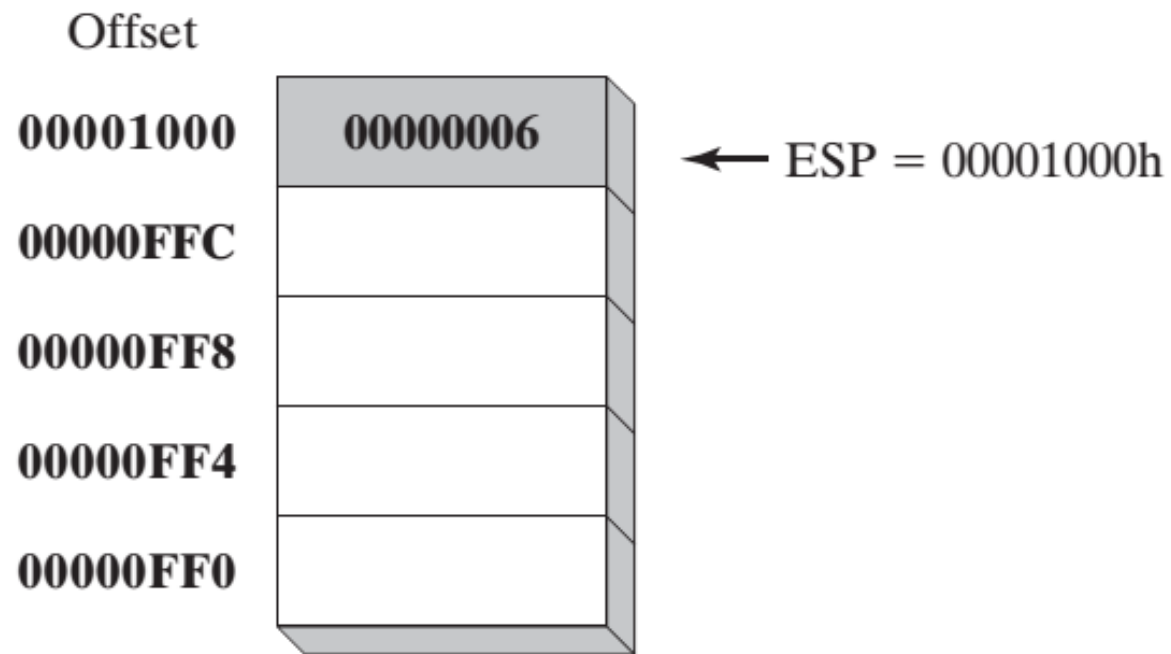
- plates are only added to the top
- plates are only removed from the top
- LIFO structure



5.1 STACK OPERATIONS

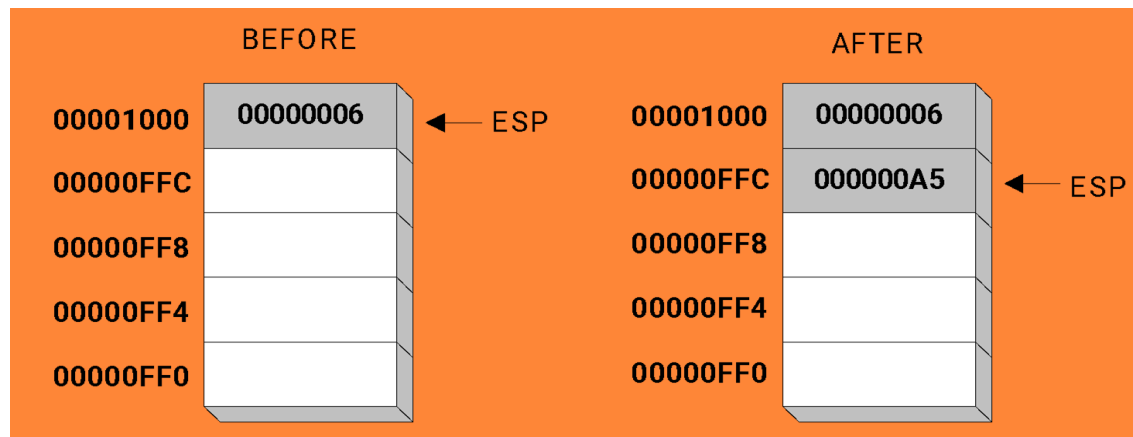
- A *stack data structure* follows the same principle as a stack of plates:
 - New values are added to the top of the stack, and existing values are removed from the top.
- A stack is also called a LIFO structure (*Last-In, First-Out*) because the last value put into the stack is always the first value taken out.
- **Runtime Stack**
 - The *runtime stack* is a memory array managed directly by the CPU, using the ESP (extended stack pointer) register, known as the *stack pointer register*.
 - ESP always points to the last value to be added to, or *pushed* on, the top of stack.
 - We rarely manipulate ESP directly; instead, it is indirectly modified by instructions such as CALL, RET, PUSH, and POP



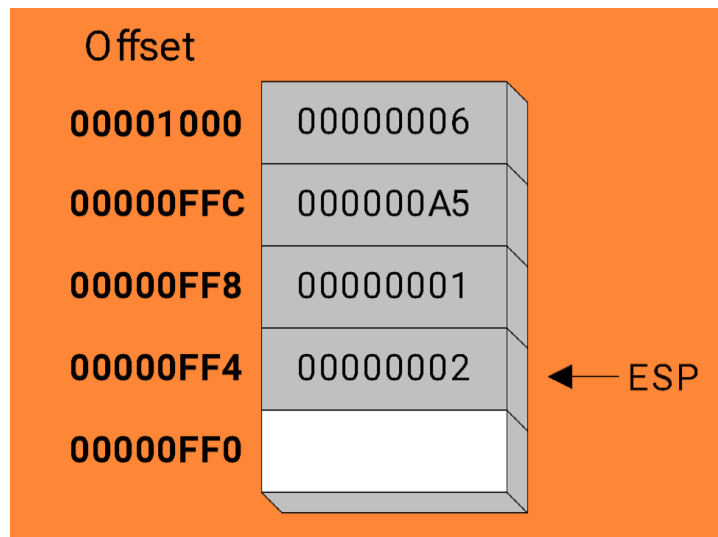


PUSH OPERATION

- A 32-bit *push operation* decrements the stack pointer by 4 and copies a value into the location in the stack pointed to by the stack pointer.



- Same stack after pushing two more integers:

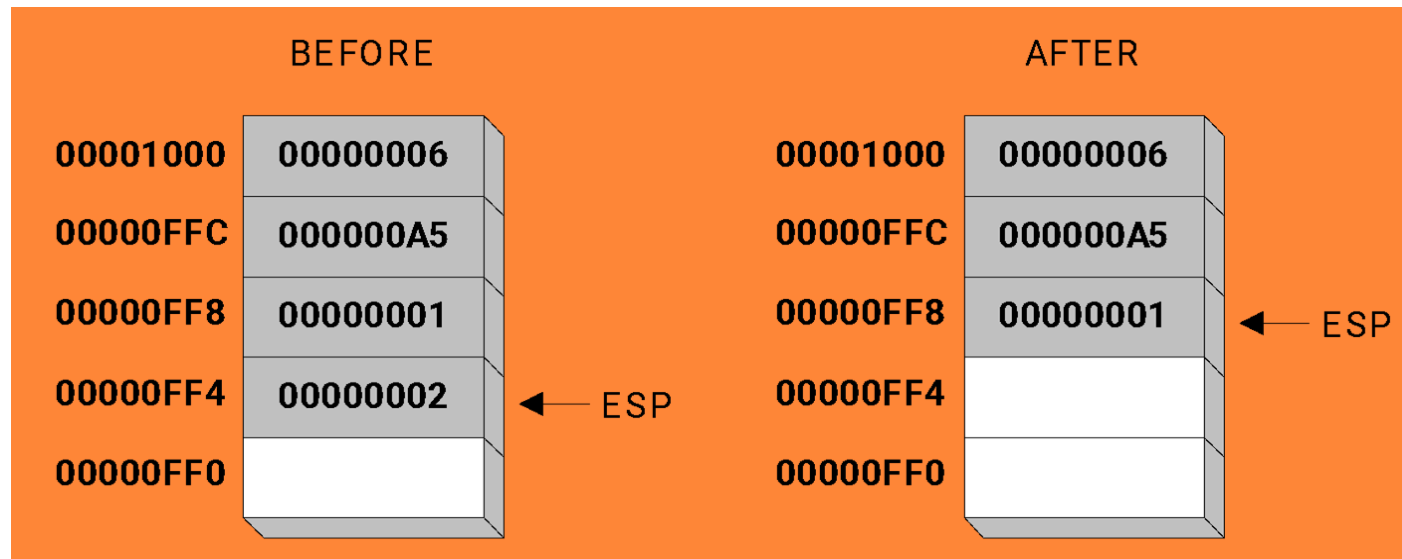


- runtime stack grows downward in memory, from higher addresses to lower addresses.



POP OPERATION

- A *pop operation* removes a value from the stack. After the value is popped from the stack, the stack pointer is incremented (by the stack element size) to point to the next-highest location in the stack.



STACK APPLICATIONS

- A stack makes a convenient temporary save area for registers when they are used for more than one purpose.
- When the CALL instruction executes, the CPU saves the current subroutine's return address on the stack.
- When calling a subroutine, you pass input values called *arguments* by pushing them on the stack.
- The stack provides temporary storage for local variables inside subroutines.



PUSH AND POP INSTRUCTIONS

- The **PUSH** instruction first decrements ESP and then copies a source operand into the stack.
 - A 16-bit operand causes ESP to be decremented by 2.
 - A 32-bit operand causes ESP to be decremented by 4.

`PUSH reg/mem16`

`PUSH reg/mem32`

`PUSH imm32`



- The **POP** instruction first copies the contents of the stack element pointed to by ESP into a 16- or 32-bit destination operand and then increments ESP.
- If the operand is 16 bits, ESP is incremented by 2; if the operand is 32 bits, ESP is incremented by 4:

`POP reg/mem16`

`POP reg/mem32`



PUSHFD , POPFD

- PUSHFD instruction pushes the 32bit EFLAGS register on the stack, and POPFD pops the stack into EFLAGS

Pushfd

Popfd

MOV instruction cannot be used to copy the flags to a variable, so PUSHFD may be the best way to save the flags

PUSHFD , POPFD

- A less error- prone way to save and restore the flags is to push them on the stack and immediately pop them into a variable

```
.data
    saveFlags Dword ?
.code
    Pushfd                ; push flag on stack
    Pop saveflags         ; copy into a variable
```

The following statement restore the flags from the same variabele

```
Push saveFlags           ; push saved flag values
Popfd                    ; copy into the flags
```

PUSHAD, PUSHA, POPAD, POPA

- The **PUSHAD** instruction pushes all of the 32-bit general-purpose registers on the stack in the following order: EAX, ECX, EDX, EBX, ESP (value before executing PUSHAD), EBP, ESI, and EDI.
- The **POPAD** instruction pops the same registers off the stack in reverse order.
- Similarly, the **PUSHA** instruction, pushes the 16-bit general-purpose registers (AX, CX, DX, BX, SP, BP, SI, DI) on the stack in the order listed. The **POPA** instruction pops the same registers in reverse.



```

.data
aName BYTE "Abraham Lincoln",0
nameSize = ($ - aName) - 1

.code
main PROC
; Push the name on the stack.
    mov     ecx,nameSize
    mov     esi,0

L1:  movzx  eax,aName[esi]           ; get character
     push  eax                     ; push on stack
     inc   esi
     loop  L1

; Pop the name from the stack, in reverse,
; and store in the aName array.
    mov     ecx,nameSize
    mov     esi,0

L2:  pop    eax                     ; get character
     mov    aName[esi],al           ; store in string
     inc    esi

     loop  L2

    INVOKE  ExitProcess,0
main ENDP
END main

```

EXAMPLE:
REVERSING A
STRING



5.2 DEFINING AND USING PROCEDURES

- A complicated problem is usually divided into separate tasks (subroutines).
 - We typically use the term *procedure* to mean a subroutine.
- A procedure is named block of statements that ends in a return statement
 - Declared using PROC and ENDP directives
- Must be assigned a name (valid identifier).

```
main PROC
```

```
.
```

```
main ENDP
```



- When you create a procedure other than your program's startup procedure, end it with a RET instruction.
 - RET forces the CPU to return to the location from where the procedure was called:

```
sample PROC
    add eax,ebx
    add eax,ecx
    ret
sample ENDP
```



- **Labels in Procedures** are visible only within the procedure in which they are declared.
 - In the following example, the label named *Destination* must be located in the same procedure as the JMP instruction:

```
jmp Destination
```

- It is possible to work around this limitation by declaring a *global label*, identified by a double colon (::) after its name:

```
Destination::
```



CALL AND RET INSTRUCTIONS

- The **CALL** instruction calls a procedure by directing the processor to begin execution at a new memory location
 - Pushes offset of next (instruction after call) on the stack
 - Copies the address of the called procedure into EIP
- The **RET** instruction returns from a procedure
 - ➡ Pops top of stack into **EIP**



CALL-RET EXAMPLE (1 OF 2)

0000025 is the offset of the instruction immediately following the CALL instruction

00000040 is the offset of the first instruction inside MySub

```
main PROC
    00000020 call MySub
    00000025 mov  eax,ebx
    .
    .
main ENDP

MySub PROC
    00000040 mov  eax,edx
    .
    .
    ret
MySub ENDP
```

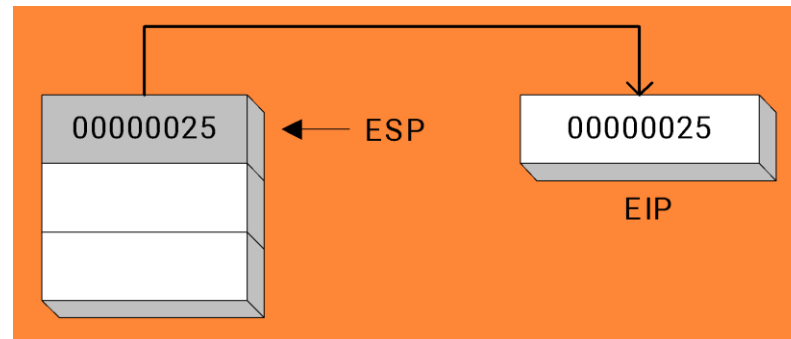


CALL-RET EXAMPLE (2 OF 2)

The CALL instruction pushes 00000025 onto the stack, and loads 00000040 into EIP



The RET instruction pops 00000025 from the stack into EIP

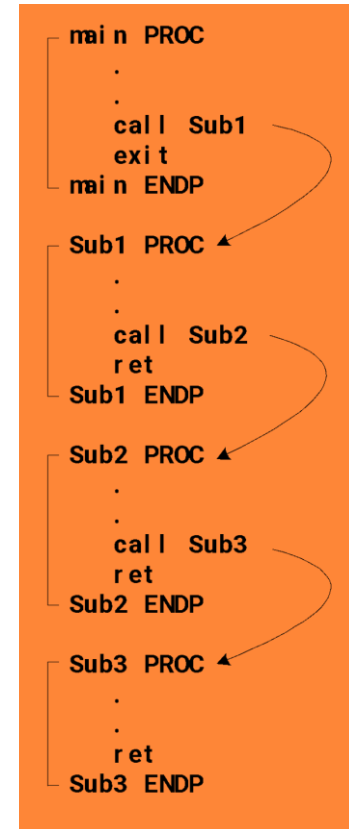
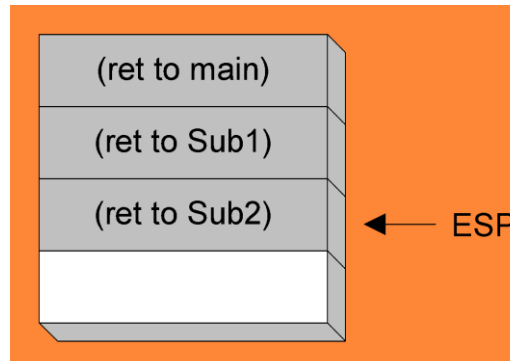


(stack shown before RET executes)



- A *nested procedure call* occurs when a called procedure calls another procedure before the first procedure returns.

By the time Sub3 is called, the stack contains all three return addresses:



In assembly language, it is common to **pass arguments inside general-purpose** registers.

```
.data
theSum  DWORD  ?
.code
main PROC
    mov     eax,10000h           ; argument
    mov     ebx,20000h           ; argument
    mov     ecx,30000h           ; argument
    call    Sumof                ; EAX = (EAX + EBX + ECX)
    mov     theSum,eax           ; save the sum

SumOf PROC
    add     eax,ebx
    add     eax,ecx
    ret
SumOf ENDP
```



TESTING THE ARRAYSUM PROCEDURE (TESTARRAYSUM.ASM)

.386

.model flat, stdcall

.stack 4096

ExitProcess PROTO, dwExitCode:DWORD

.data

array DWORD 10000h,20000h,30000h,40000h,50000h

theSum DWORD ?

.code

main PROC

mov esi,OFFSET array ; ESI points to array

mov ecx,LENGTHOF array ; ECX = array count

call ArraySum ; calculate the sum

mov theSum,eax ; returned in EAX

INVOKE ExitProcess,0

main ENDP



TESTING THE ARRAYSUM PROCEDURE (TESTARRAYSUM.ASM)

```
;-----; ArraySum  
; Calculates the sum of an array of 32-bit integers.  
; Receives: ESI = the array offset  
; ECX = number of elements in the array  
; Returns: EAX = sum of the array elements  
;-----
```

ArraySum PROC

push esi ; save ESI, ECX

push ecx

mov eax,0 ; set the sum to zero

L1:

add eax,[esi] ; add each integer to sum

add esi,TYPE DWORD ; point to next integer

loop L1

Pop ecx

Pop esi

Ret

ArraySum ENDP

End main



SAVING AND RESTORING REGISTERS

In the ArraySum example,

ECX and ESI were pushed on the stack at the beginning of the procedure and popped at the end. This action is typical of most procedures that modify registers.

- Always save and restore registers that are modified by a procedure so the calling program can be sure that none of its own register values will be overwritten.
- The exception to this rule pertains to registers used as return values, usually EAX. Do not push and pop them



USES OPERATOR

USES tells the assembler to do two things:

- First, generate PUSH instructions that save the registers on the stack at the beginning of the procedure.
- Second, generate POP instructions that restore the register values at the end of the procedure.

```
ArraySum PROC USES esi ecx
```

```
mov eax,0 ; set the sum to zero
```

```
L1:
```

```
add eax,[esi] ; add each integer to sum
```

```
add esi,TYPE DWORD ; point to next integer
```

```
loop L1 ; repeat for array size
```

```
ret ; sum is in EAX
```

```
ArraySum ENDP
```



USES OPERATOR

```
ArraySum PROC USES esi ecx  
mov eax,0 ; set the sum to zero  
L1:  
add eax,[esi]  
add esi,TYPE DWORD  
loop L1 ; repeat for array size  
ret ; sum is in EAX  
ArraySum ENDP
```

Generated By Assembler

```
ArraySum PROC  
push esi  
push ecx  
mov eax,0 ; set the sum to zero  
L1:  
add eax,[esi]  
add esi,TYPE DWORD ;  
loop L1 ; repeat for array size  
pop ecx  
pop esi  
ret
```

```
ArraySum ENDP
```



SUMMARY

- Stack
- Stack Operations
 - ➞ ○ PUSH
 - ➞ ○ POP
- Defining and using Procedures
- Call and Ret instructions

